CEE6501 — Lecture 9.1

Support Settlement

Learning Objectives

By the end of this lecture, you will be able to:

- Model support settlements within the Direct Stiffness Method
- Derive the modified partitioned equations when $u_r \neq 0$
- Interpret settlement using imaginary restraints + superposition
- Relate settlement to an equivalent joint load
- Correctly compute member forces and reactions

Agenda

- **Part 1** — Conceptual Framework
- **Part 2** — Global Equations
- **Part 3** — Support Settlement, DSM Implementation
- **Part 4** — Truss Example

Part 1 — Conceptual Framework

Where Support Displacements Come From

In real structures, supports do not always remain perfectly fixed.

Small prescribed displacements may occur due to:

- Weak or compressible foundations
- Soil consolidation
- Differential settlement
- Fabrication tolerances or lack-of-fit

These are **imposed displacements**, not applied forces.

In DSM language:

$$u_r \neq 0$$

Core Question

How do we include prescribed support displacements within a force-based matrix framework?

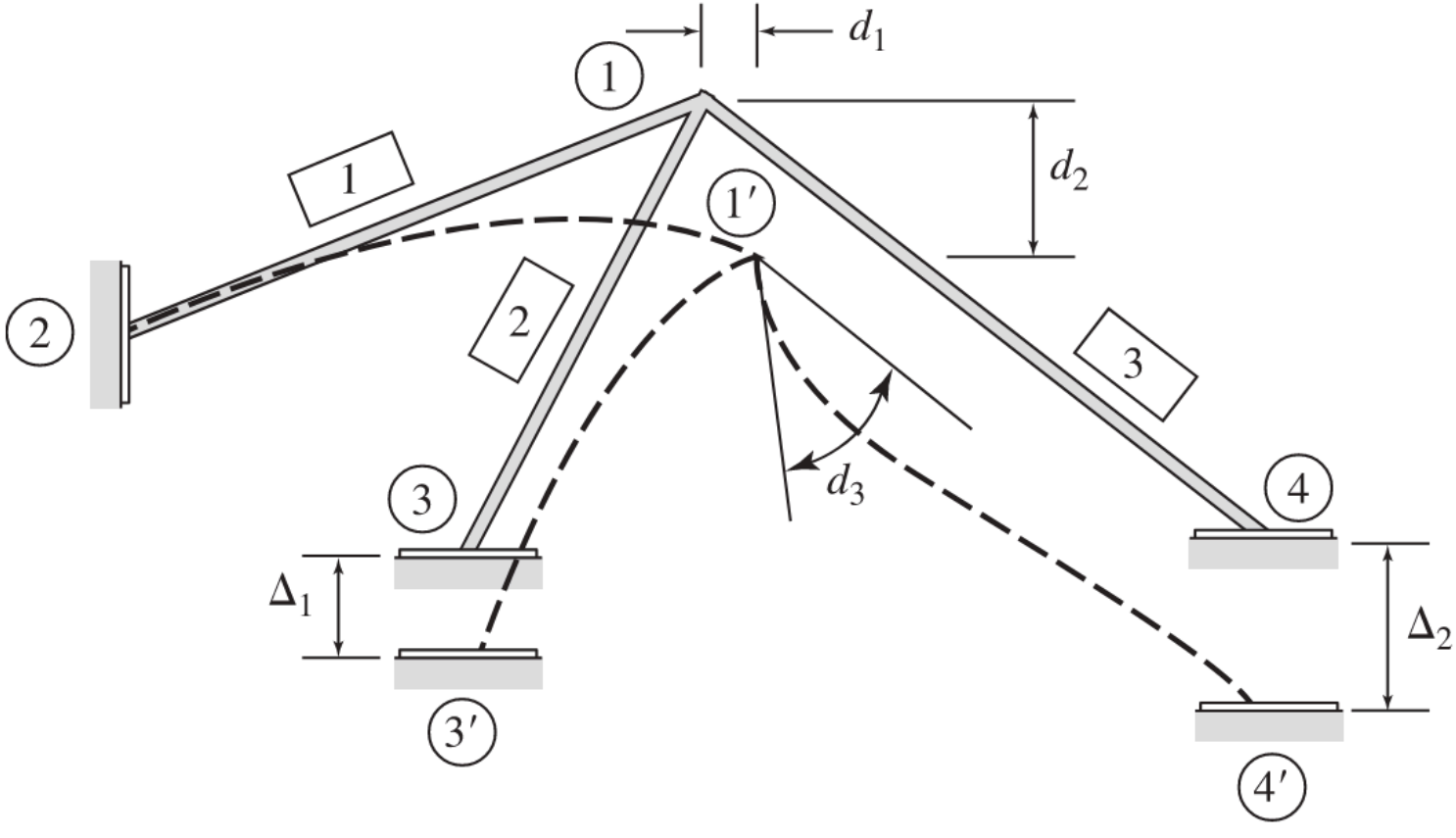
DSM solves

$$Ku = F$$

Settlement does not directly enter as a force.

So we need a way to convert:

imposed displacements → equivalent forces


d_1
1
1
d_2
1′
2
2
3
d_3
4
3
Δ_1
Δ_2
3′
4′

Key Idea

We use an imaginary fully restrained structure.

1. Restrain all joints.
2. Impose the prescribed settlements

$$\Delta_1,\ \Delta_2,\ \ldots$$

The fully fixed structure develops **fixed-joint forces**

$$F_1^S,\ F_2^S,\ F_3^S, \ldots$$

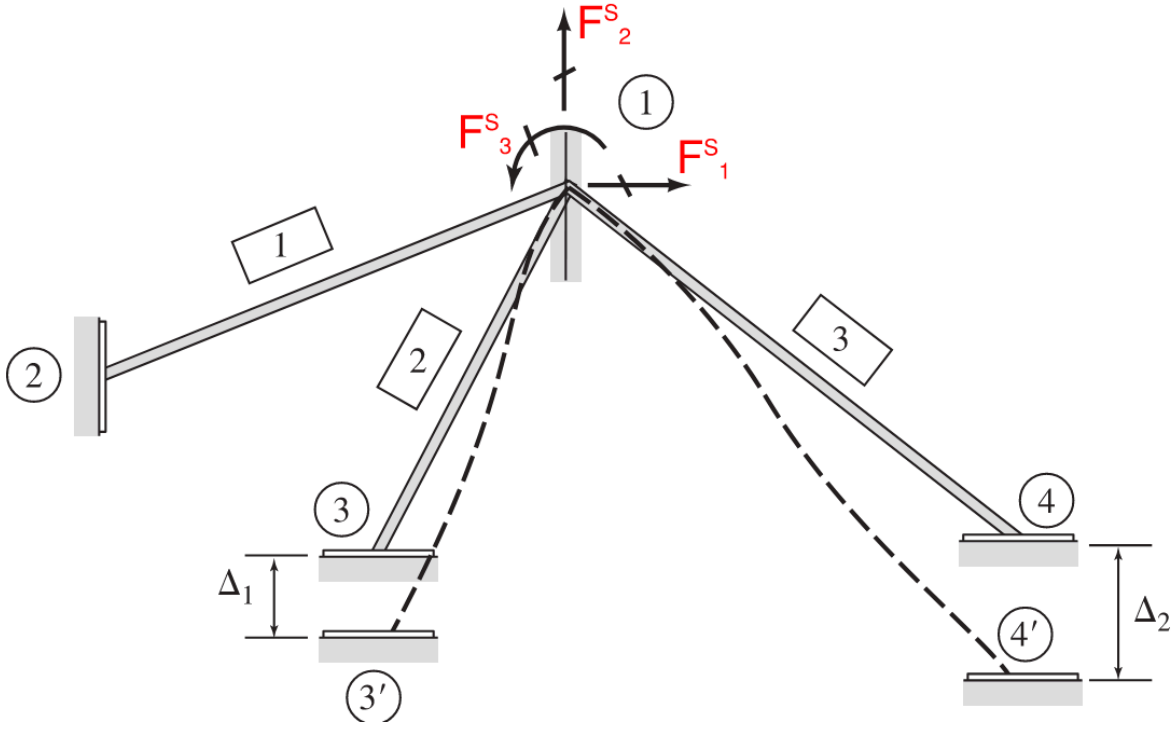
These are reactions required to enforce compatibility through superposition.

Superposition Process

We represent the actual system as:

$$
\begin{aligned}
&\text{Actual frame with settlement} \\
&= \\
&(1)\ \text{Imaginary Fully fixed frame with settlement, develops } F^S \\
&+ \\
&(2)\ \text{Real frame subjected to } -F^S
\end{aligned}
$$

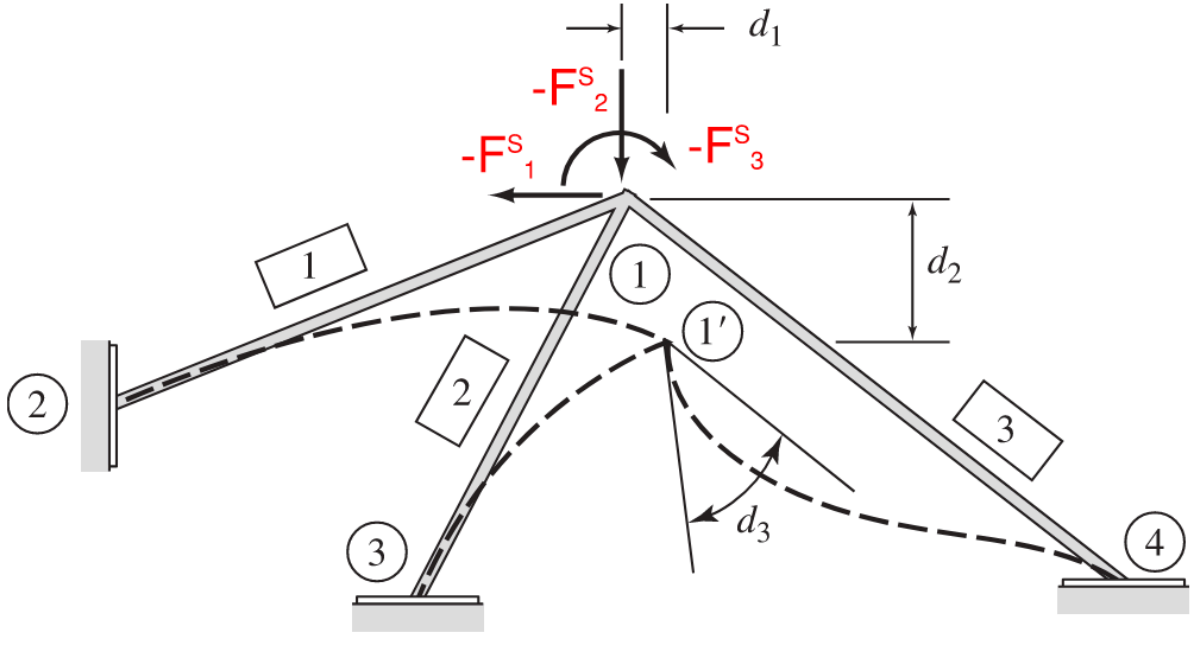
(1) Imaginary Fully fixed Frame with Settlement



- Joints restrained
- Settlements imposed
- Fixed-joint forces develop

These forces represent the structural resistance to the imposed support movements.

(2) Real Frame Subjected to $-\boldsymbol{F}^S$



What Does This Mean Physically?

- The fully fixed system develops forces F^S
- Applying their negatives to the real frame cancels out the artificial restraints
- What remains is the true compatibility-driven response

The negatives of the fixed-joint forces therefore act as:

Equivalent joint loads due to settlement

These loads produce the same joint displacements as the actual support settlements.

Part 2 — Global Equations

Global Equilibrium

The assembled system (including fixed-end forces) is

$$Ku = F - F^F$$

Partition the system:

$$\begin{bmatrix} K_{ff} & K_{fr} \\ K_{rf} & K_{rr} \end{bmatrix} \begin{Bmatrix} u_f \\ u_r \end{Bmatrix} = \begin{Bmatrix} F_f \\ F_r \end{Bmatrix} - \begin{Bmatrix} F_f^F \\ F_r^F \end{Bmatrix}$$

Normally:

$$u_r = 0$$

With support settlement:

$$u_r \neq 0$$

Free DOFs Equation

From the first block row:

$$K_{ff}u_f + K_{fr}u_r = F_f - F_f^F$$

Rearranging:

$$K_{ff}u_f = F_f - F_f^F - K_{fr}u_r$$

Must calculate $K_{fr}u_r$

Physical Interpretation of $K_{fr}\,u_r$

What does the term $K_{fr}u_r$ represent?

- u_r = **prescribed displacements** at restrained DOFs (e.g., support settlements).
- $K_{fr}u_r$ = **forces induced at the free DOFs** by those imposed displacements.

Physically, this term represents the forces that would develop in the **free DOFs if they were temporarily held fixed while the supports moved**.

The K_{fr} Terms

$$\boldsymbol{K} = \left[\begin{array}{c:c} \boldsymbol{K}_{ff} & \boldsymbol{K}_{fr} \\ \hdashline \boldsymbol{K}_{rf} & \boldsymbol{K}_{rr} \end{array}\right] = \left[\begin{array}{cc:cccc} K_{33} & K_{35} & \boxed{K_{31}} & \boxed{K_{34}} & \boxed{K_{32}} & \boxed{K_{36}} \\ K_{53} & K_{55} & \boxed{K_{51}} & \boxed{K_{54}} & \boxed{K_{52}} & \boxed{K_{56}} \\ \hdashline K_{13} & K_{15} & K_{11} & K_{14} & K_{12} & K_{16} \\ K_{43} & K_{45} & K_{41} & K_{44} & K_{42} & K_{46} \\ K_{23} & K_{25} & K_{21} & K_{24} & K_{22} & K_{26} \\ K_{63} & K_{65} & K_{61} & K_{64} & K_{62} & K_{66} \end{array}\right]$$

Recall the definition of the stiffness coefficients:

$$k_{ij} = \text{force at DOF } i \text{ caused by a unit displacement at DOF } j, \\ \text{with all other DOFs fixed.}$$

In this example matrix (taken from Lecture 4):

- **Free DOFs:** $3, 5$ → i in K_{fr}
- **Restrained DOFs:** $1, 2, 4, 6$ → j in K_{fr}

Each column of K_{fr} represents the **forces at the free DOFs caused by a unit displacement at a restrained DOF**.

Instead of a unit displacement, we now apply the **actual prescribed displacement** (e.g., a support settlement).

The product, $K_{fr}u_r$, therefore gives the **forces induced at the free DOFs by the imposed support movements**.

These forces are analogous to the **fixed-joint forces**, F_f^S, because they correspond to the forces that would develop if the structure were **temporarily fixed at all DOFs** while the support DOFs were displaced.

Thus,

$$F_f^S = K_{fr}u_r$$

is the expression for the **fixed-joint force vector**, which is then **subtracted from the real system** to remove the artificial restraint in the superposition process.

Reactions

From the second block row of the partitioned system:

$$F_r = K_{rf} u_f + K_{rr} u_r + F_r^F$$

The term $K_{rr} u_r$ represents the **reactions generated directly by the imposed support displacements**.

When the supports move by u_r, the structure must develop forces to **satisfy displacement compatibility** at the restrained DOFs.

Big Picture Connection

Part 1 (Conceptual):

- Settlement → imaginary fully fixed structure
- Develop fixed-joint forces F^S
- Apply $-F^S$ to the real structure

Part 2 (Matrix Form):

- Settlement enters through u_r
- Fixed-joint forces to be subtracted appear as

$$F^S = K_{fr} u_r$$

No new theory is needed!

The superposition argument and the matrix formulation are the same mechanics as we used in definition FEFs in Lecture 6.

Part 3 — Support Settlement DSM Implementation

Code implementation steps

1. Assemble global K
2. Assemble applied load vector, F
3. Assemble fixed end force vector, F^F
4. Insert prescribed settlements into u_r
5. Partition matrices
6. Solve

$$u_f = K_{ff}^{-1}(F_f - F_f^F - K_{fr}u_r)$$

6. Compute reactions

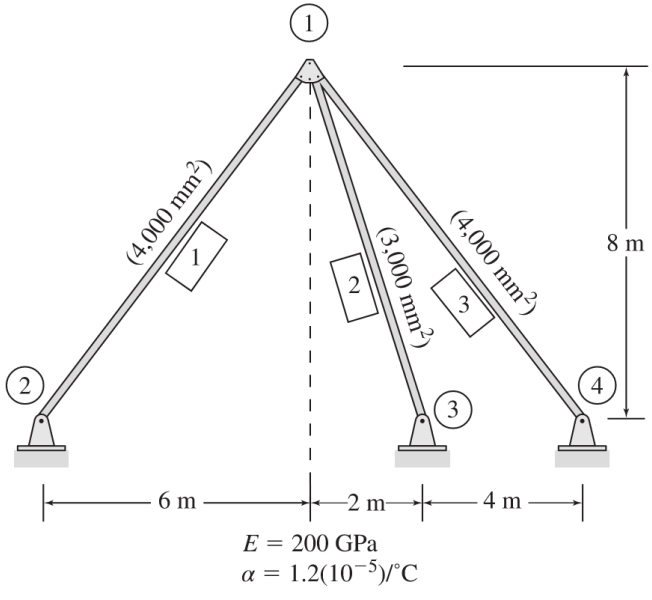
$$F_r = K_{rf}u_f + K_{rr}u_r + F_r^F$$

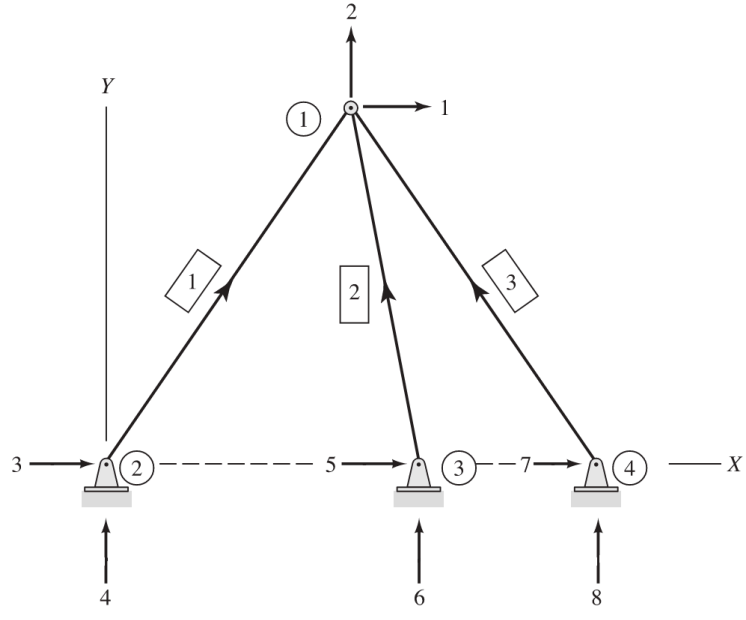
7. Backward pass to recover element level forces

Part 4 — Truss Example

Example Problem (7.2 Kassimali)

Same truss structure we looked at back in Lecture 4.





Determine the joint displacements, member axial forces, and support reactions for the plane truss due to a settlement of 10 mm of support 4.

In [262]:

```python
# -------------------------
# Import Libraries and Helper Functions
# -------------------------
# This slide imports the required Python libraries and adds the
# course helper-code directory to the Python path.

import sys
import os
import math
import numpy as np

# Get current notebook directory
current_dir = os.getcwd()

# Go up two levels, then into Code/L8
helpers_path = os.path.abspath(
    os.path.join(current_dir, "..", "..", "Code", "L8")
)

sys.path.append(helpers_path)
```

In [263]:
```
from helper_funcs_dsm import (
    assemble_global_stiffness_and_fef,
    partition_system,
    assemble_global_displacements,
    assemble_global_forces
)
from helper_funcs_output import (
    print_element_truss,
    print_dsm_results,
    print_matrix_scaled,
)

# -------------------------
# Problem data (hard-coded)
# -------------------------
E = 200e6           # kN/m^2
A1 = 0.004 # m^2
L1 = math.sqrt(6**2 + 8**2)

A2 = 0.003 # m^2
L2 = math.sqrt(2**2 + 8**2)

A3 = 0.004 # m^2
L3 = math.sqrt(6**2 + 8**2)
```

In [264]:

```
# -------------------------
# Member 1, hard-coded
# -------------------------
theta = math.atan2(8,6)  # radians
c = np.cos(theta)
s = np.sin(theta)

factor = E * A1 / L1

k1 = factor * np.array([
    [ 1., 0., -1., 0.],
    [ 0., 0.,  0., 0.],
    [-1., 0.,  1., 0.],
    [ 0., 0.,  0., 0.]
], dtype=float)

# Local fixed-end vector 0 for Trusses
Qf1 = np.zeros(4, dtype=float)

T1 = np.array([
    [ c,  s, 0.,  0.],
    [-s,  c, 0.,  0.],
    [ 0.,  0., c,  s],
    [ 0.,  0.,-s,  c]
], dtype=float)

# Mapping
m1 = np.array([3,4,1,2])
```

In [265]:

```
# -------------------------
# Member 2, hard-coded
# -------------------------
theta = math.atan2(8,-2)
c = np.cos(theta)
s = np.sin(theta)

factor = E * A2 / L2

k2 = factor * np.array([
    [ 1., 0., -1., 0.],
    [ 0., 0.,  0., 0.],
    [-1., 0.,  1., 0.],
    [ 0., 0.,  0., 0.]
], dtype=float)

# Local fixed-end vector 0 for Trusses
Qf2 = np.zeros(4, dtype=float)

T2 = np.array([
    [ c,  s, 0.,  0.],
    [-s,  c, 0.,  0.],
    [ 0.,  0., c,  s],
    [ 0.,  0.,-s,  c]
], dtype=float)

m2 = np.array([5,6,1,2])
```

In [266]:

```
# -------------------------
# Member 3, hard-coded
# -------------------------
theta = math.atan2(8,-6)
c = np.cos(theta)
s = np.sin(theta)

factor = E * A3 / L3

k3 = factor * np.array([
    [ 1., 0., -1., 0.],
    [ 0., 0.,  0., 0.],
    [-1., 0.,  1., 0.],
    [ 0., 0.,  0., 0.]
], dtype=float)

# Local fixed-end vector 0 for Trusses
Qf3 = np.zeros(4, dtype=float)

T3 = np.array([
    [ c,  s, 0.,  0.],
    [-s,  c, 0.,  0.],
    [ 0.,  0., c,  s],
    [ 0.,  0.,-s,  c]
], dtype=float)

m3 = np.array([7,8,1,2])
```

In [267]:

```
# -------------------------
# Initialize Global System
# -------------------------
# This slide sets up the global variables needed for assembly of the
# Direct Stiffness Method system.

k_list = [k1, k2, k3]
T_list = [T1, T2, T3]
Qf_list = [Qf1, Qf2, Qf3]
map_list = [m1, m2, m3]   # 1-based DOF maps

# Global system size (still using 1-based maps here)
ndof = int(np.max(np.concatenate(map_list)))

# Initialize Applied Load
F_global = np.zeros(ndof, dtype=float)

# Initialize Global Displacement
u_global = np.zeros(ndof, dtype=float)
```

In [268]:

```
# -------------------------
# User-Specified Boundary Conditions
# -------------------------
# This slide defines the external loads and support conditions.

# Global Displacement (0-indexed)
u_global[8-1] = -0.01 #m

dof_restrained_1based = np.sort(
    np.array([3, 4, 5, 6, 7, 8], dtype=int)
)
```

In [269]:

```
# -------------------------
# Assemble Global Stiffness System
# -------------------------
# This step assembles the global stiffness matrix and the global
# fixed-end force vector from all elements.

K_global, F_fef_global = assemble_global_stiffness_and_fef(
    ndof,
    k_list,
    T_list,
    Qf_list,
    map_list
    )
```

In [270]:
```
# -------------------------
# Display Global Stiffness Matrix
# -------------------------
# Print the assembled global stiffness matrix in a readable format.
# The matrix is scaled for clearer presentation in the slides.

print_matrix_scaled(K_global, scale=1000, decimals=2, col_width=6)
```

```
K = 1e+03 ×
01 |  61.88 -17.12 -28.80 -38.40  -4.28  17.12 -28.80  38.40
02 | -17.12 170.88 -38.40 -51.20  17.12 -68.48  38.40 -51.20
03 | -28.80 -38.40  28.80  38.40   0.00   0.00   0.00   0.00
04 | -38.40 -51.20  38.40  51.20   0.00   0.00   0.00   0.00
05 |  -4.28  17.12   0.00   0.00   4.28 -17.12   0.00   0.00
06 |  17.12 -68.48   0.00   0.00 -17.12  68.48   0.00   0.00
07 | -28.80  38.40   0.00   0.00   0.00   0.00  28.80 -38.40
08 |  38.40 -51.20   0.00   0.00   0.00   0.00 -38.40  51.20
```

In [271]:

```
# -------------------------
# Partition Global System
# -------------------------
# The global stiffness equation is partitioned into free and restrained
# degrees of freedom based on the support conditions.

(
    K_ff,
    K_fr,
    K_rf,
    K_rr,
    f_f,
    f_r,
    u_r,
    f_fef_f,
    f_fef_r,
    free_dofs,
    restrained_dofs,
) = partition_system(
    K_global,
    F_global,
    u_global,
    F_fef_global,
    dof_restrained_1based
)
```

In [272]:

```
# -------------------------
# Solve Partitioned System
# -------------------------
# Solve for the unknown displacements at the free DOFs,
# compute the reactions at restrained DOFs, and then
# reconstruct the complete global displacement and force vectors.

rhs = f_f - f_fef_f - K_fr @ u_r
u_f = np.linalg.solve(K_ff, rhs)

F_r = K_rf @ u_f + K_rr @ u_r + f_fef_r

u_global = assemble_global_displacements(
    u_f,
    u_r,
    free_dofs,
    restrained_dofs
    )

f_global_complete = assemble_global_forces(
    f_f,
    F_r,
    free_dofs,
    restrained_dofs
    )
```

In [273]:

```
# -------------------------
# Display DSM Results
# -------------------------
# Print the final global displacement vector and force vector.

print_dsm_results(
    u_global,
    f_global_complete,
    dof_restrained_1based,
    disp_in_mm=True
)
```

```
  DOF  Type Status  Disp (mm / rad)  Load (kN / kN·m)
    1   u_x   Free            5.530             0.000
    2   u_y   Free           -2.442             0.000
    3 theta  Fixed            0.000           -65.479
    4   u_x  Fixed            0.000           -87.306
    5   u_y  Fixed            0.000           -65.479
    6 theta  Fixed            0.000           261.917
    7   u_x  Fixed            0.000           130.958
    8   u_y  Fixed          -10.000          -174.611
```

Member-Level Solution (Local)

Then the backward pass goes extract u_global for the element

$$\mathbf{u}_i \leftarrow \mathbf{u}_{\text{global}}$$

Convert global DOFs to the element's local coordinate system:

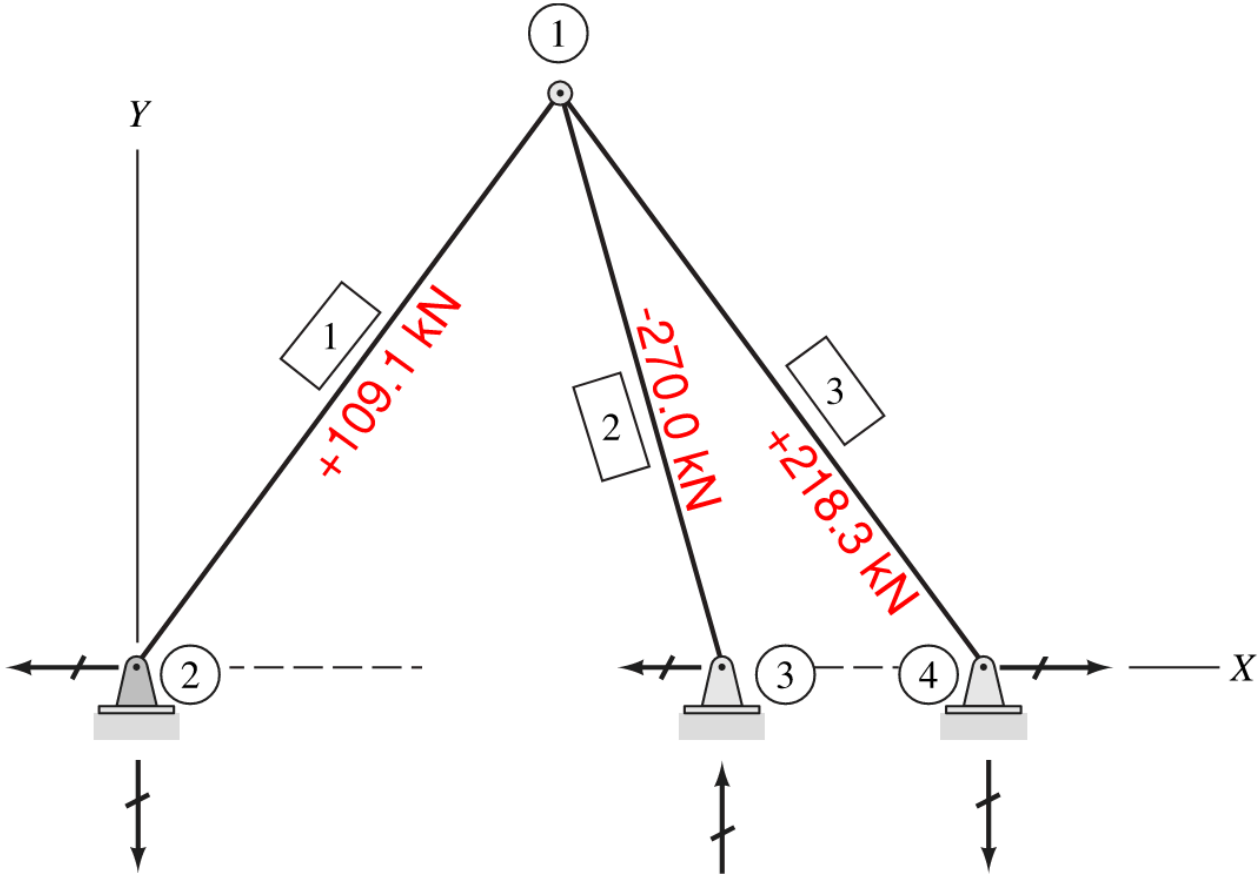
$$\mathbf{v}_i = \mathbf{T}_i \, \mathbf{u}_i$$

Local element forces are the sum of displacement-related forces and FEFs:

$$\mathbf{q}_i = \mathbf{k}_{\ell,i} \, \mathbf{v}_i + \mathbf{q}_i^F$$

In [274]:
```
# -------------------------
# Display Element Results
# -------------------------
# Compute and print the element-level results for each member.

print_element_truss(1, u_global, m1, T1, k1, Qf1, disp_in_mm=True, dec=2
print_element_truss(2, u_global, m2, T2, k2, Qf2, disp_in_mm=True, dec=2
print_element_truss(3, u_global, m3, T3, k3, Qf3, disp_in_mm=True, dec=2
```

```
E1 (Truss)
u_global [mm]: [0.00, 0.00, 5.53, -2.44]
u_local  [mm]: [0.00, 0.00, 1.36, -5.89]
q_local  [kN]: [-109.13, 0.00, 109.13, 0.00]
N (tension +) = 109.13 kN


E2 (Truss)
u_global [mm]: [0.00, 0.00, 5.53, -2.44]
u_local  [mm]: [0.00, 0.00, -3.71, -4.77]
q_local  [kN]: [269.98, 0.00, -269.98, 0.00]
N (tension +) = -269.98 kN


E3 (Truss)
u_global [mm]: [0.00, -10.00, 5.53, -2.44]
u_local  [mm]: [-8.00, 6.00, -5.27, -2.96]
q_local  [kN]: [-218.26, 0.00, 218.26, 0.00]
N (tension +) = 218.26 kN
```


1
Y
1
+109.1 kN
2
-270.0 kN
3
+218.3 kN
2
3
4
X

Member-Level Solution (Global)

Then transform **member end forces in local coordinates** to the **global coordinate system** using $f = T^T q$

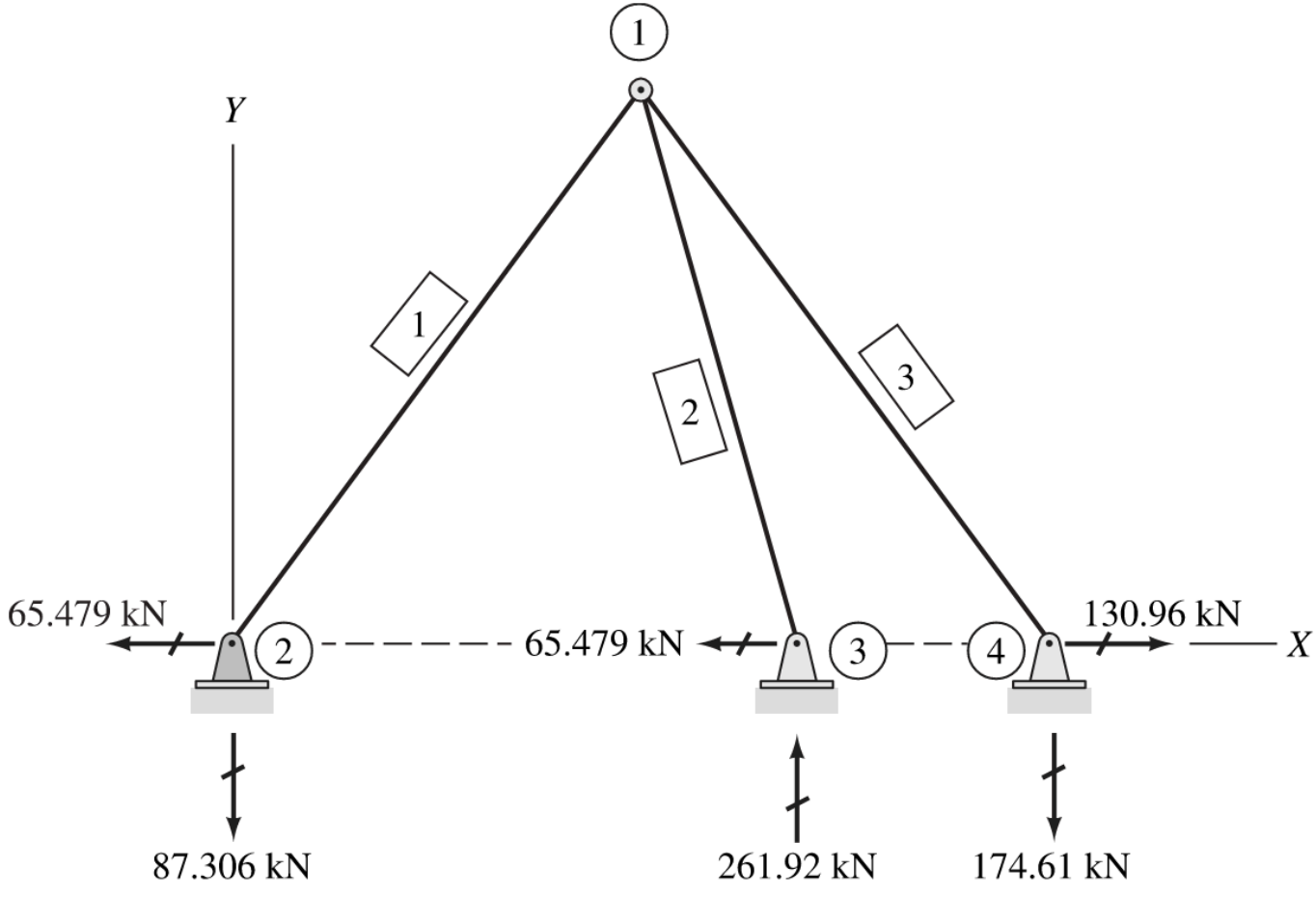
The resulting global force components provide the **reactions at the supports** due to the prescribed support displacement.

In [275]:
```
idx = m1 - 1
u = u_global[idx]
v = T1 @ u
q = k1 @ v + Qf1
f = T1 @ q

print(f)
```

```
[-65.47923942  87.30565256  65.47923942 -87.30565256]
```

All elements start at supports and end at top node.

First two terms in each element's f-vector give support forces:



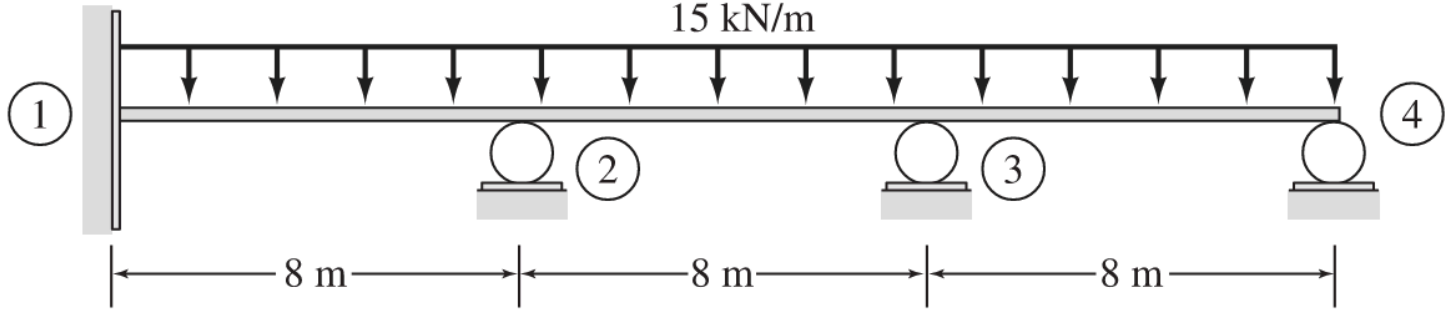
Part 5 — In-Class Exercise

Open `InClass_Excercise.ipynb` in the `../../Code/L9` folder

Part 6 — Solution In-Class Exercise

Determine the joint displacements, member end forces, and support reactions for a continuous beam, due to the combined effect of the uniformly distributed load shown and the settlements of 45 mm and 15 mm, respectively, of supports 3 and 4.

Using the DSM, model the structure as frame elements, not beam elements, you should get the same answer as the textbook.



EI = constant
E = 70 GPa
$I = 102(10^6)\ \text{mm}^4$

In [276]:

```python
# -------------------------
# Import Libraries and Helper Functions
# -------------------------
# This slide imports the required Python libraries and adds the
# course helper-code directory to the Python path.

import sys
import os
import numpy as np
import pandas as pd

# Get current notebook directory
current_dir = os.getcwd()

# Go up two levels, then into Code/L8
helpers_path = os.path.abspath(
    os.path.join(current_dir, "..", "..", "Code", "L8")
)

sys.path.append(helpers_path)
```

In [277]:

```
import numpy as np

from helper_funcs_dsm import (
    assemble_global_stiffness_and_fef,
    partition_system,
    assemble_global_displacements,
    assemble_global_forces
)
from helper_funcs_output import (
    print_element,
    print_dsm_results,
    print_matrix_scaled
)
```

In [278]:
```
# -------------------------
# Problem data (hard-coded)
# -------------------------
E = 200e6           # kPa
I = 102e-6          # m^4
L = 8.0             # m
```

In [279]:

```
# -------------------------
# Member 1
# -------------------------
k1 = np.array([
    [162500.,     0.,       0.,  -162500.,     0.,       0.],
    [    0.,   167.34,   669.38,      0.,  -167.34,   669.38],
    [    0.,   669.38,  3570.00,      0.,  -669.38,  1785.00],
    [-162500.,    0.,       0.,   162500.,     0.,       0.],
    [    0.,  -167.34,  -669.38,      0.,   167.34,  -669.38],
    [    0.,   669.38,  1785.00,      0.,  -669.38,  3570.00]
], dtype=float)

# Transformation matrix for theta=0 (identity matrix)
T1 = np.eye(6, dtype=float)
# Mapping
m1 = np.array([1,2,3,4,5,6])
```

In [280]:
```
# Local fixed-end vector for member 1 (given by the example result)
Qf1 = np.array([0., 60., 80., 0., 60., -80.], dtype=float)
```

In [281]:

```
# -------------------------
# Member 2
# -------------------------
k2 = np.array([
    [162500.,     0.,       0.,  -162500.,     0.,       0.],
    [    0.,   167.34,   669.38,      0.,  -167.34,   669.38],
    [    0.,   669.38,  3570.00,      0.,  -669.38,  1785.00],
    [-162500.,    0.,       0.,   162500.,     0.,       0.],
    [    0.,  -167.34,  -669.38,      0.,   167.34,  -669.38],
    [    0.,   669.38,  1785.00,      0.,  -669.38,  3570.00]
], dtype=float)

# Transformation matrix for theta=0 (identity matrix)
T2 = np.eye(6, dtype=float)
# Mapping
m2 = np.array([4,5,6,7,8,9])
```

In [282]:
```
# Local fixed-end vector for member 2
Qf2 = np.array([0., 60., 80., 0., 60., -80.], dtype=float)
```

In [283]:

```
# -------------------------
# Member 3
# -------------------------
k3 = np.array([
    [162500.,     0.,       0.,  -162500.,     0.,       0.],
    [    0.,   167.34,   669.38,      0.,  -167.34,   669.38],
    [    0.,   669.38,  3570.00,      0.,  -669.38,  1785.00],
    [-162500.,    0.,       0.,   162500.,     0.,       0.],
    [    0.,  -167.34,  -669.38,      0.,   167.34,  -669.38],
    [    0.,   669.38,  1785.00,      0.,  -669.38,  3570.00]
], dtype=float)

# Transformation matrix for theta=0 (identity matrix)
T3 = np.eye(6, dtype=float)
# Mapping
m3 = np.array([7,8,9,10,11,12])
```

In [284]:
```
# Local fixed-end vector for member 3
Qf3 = np.array([0., 60., 80., 0., 60., -80.], dtype=float)
```

In [285]:

```
# -------------------------
# Initialize Global System
# -------------------------
# This slide sets up the global variables needed for assembly of the
# Direct Stiffness Method system.

k_list = [k1, k2, k3]
T_list = [T1, T2, T3]
Qf_list = [Qf1, Qf2, Qf3]
map_list = [m1, m2, m3]   # 1-based DOF maps

# Global system size (still using 1-based maps here)
ndof = int(np.max(np.concatenate(map_list)))

# Initialize Applied Load
F_global = np.zeros(ndof, dtype=float)

# Initialize Global Displacement
u_global = np.zeros(ndof, dtype=float)
```

In [286]:

```
# -------------------------
# User-Specified Boundary Conditions
# -------------------------
# This slide defines the external loads and support conditions.

# Applied Load
# F_global[0] = 0.0

# Global Displacement (0-indexed)
u_global[8-1] = -0.045 #m
u_global[11-1] = -0.015 #m

# Restrained DOFs
dof_restrained_1based = np.array([1, 2, 3, 5, 8, 11], dtype=int)
```

In [287]:

```
# -------------------------
# Assemble Global Stiffness System
# -------------------------
# This step assembles the global stiffness matrix and the global
# fixed-end force vector from all elements.

K_global, F_fef_global = assemble_global_stiffness_and_fef(
    ndof,
    k_list,
    T_list,
    Qf_list,
    map_list
    )
```

In [288]:

```
# -------------------------
# Display Global Stiffness Matrix
# -------------------------
# Print the assembled global stiffness matrix in a readable format.
# The matrix is scaled for clearer presentation in the slides.

# print_matrix_scaled(K_global, scale=1000, decimals=2, col_width=6)
```

In [289]:

```
# -------------------------
# Assemble Global Stiffness System
# -------------------------
# This step assembles the global stiffness matrix and the global
# fixed-end force vector from all elements.

K_global, F_fef_global = assemble_global_stiffness_and_fef(
    ndof,
    k_list,
    T_list,
    Qf_list,
    map_list
    )
```

In [290]:

```
# -------------------------
# Partition Global System
# -------------------------
# The global stiffness equation is partitioned into free and restrained
# degrees of freedom based on the support conditions.

(
    K_ff,
    K_fr,
    K_rf,
    K_rr,
    f_f,
    f_r,
    u_r,
    f_fef_f,
    f_fef_r,
    free_dofs,
    restrained_dofs,
) = partition_system(
    K_global,
    F_global,
    u_global,
    F_fef_global,
    dof_restrained_1based
)
```

In [291]:
```
# -------------------------
# Solve Partitioned System
# -------------------------
# Solve for the unknown displacements at the free DOFs,
# compute the reactions at restrained DOFs, and then
# reconstruct the complete global displacement and force vectors.

rhs = f_f - f_fef_f - K_fr @ u_r
u_f = np.linalg.solve(K_ff, rhs)

F_r = K_rf @ u_f + K_rr @ u_r + f_fef_r

u_global = assemble_global_displacements(
    u_f,
    u_r,
    free_dofs,
    restrained_dofs
    )

f_global_complete = assemble_global_forces(
    f_f,
    F_r,
    free_dofs,
    restrained_dofs
    )
```

In [292]:
```
# -------------------------
# Display DSM Results
# -------------------------
# Print the final global displacement vector and force vector.

print_dsm_results(
    u_global,
    f_global_complete,
    dof_restrained_1based,
    disp_in_mm=True
)
```

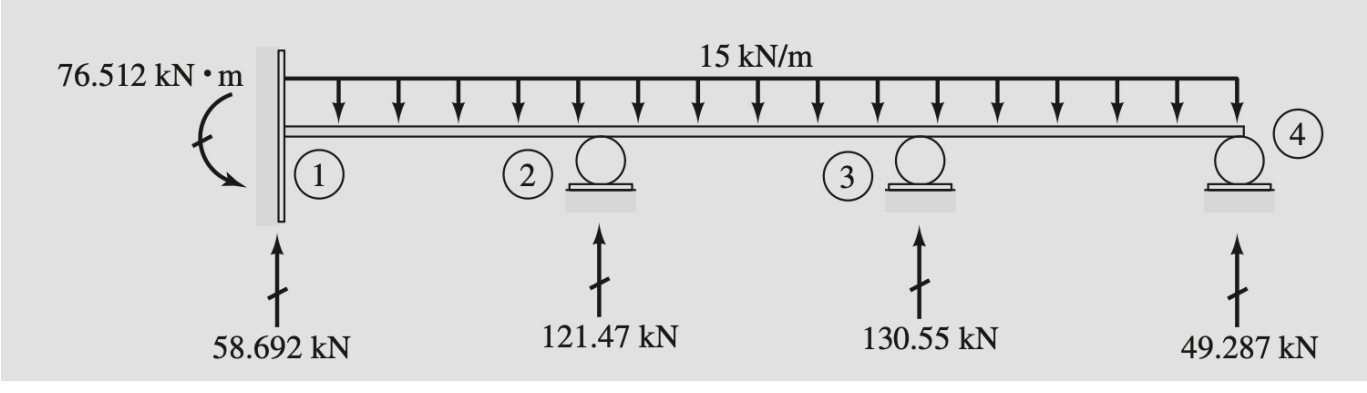
```
  DOF  Type Status  Disp (mm / rad)  Load (kN / kN·m)
    1   u_x  Fixed            0.000             0.000
    2   u_y  Fixed            0.000            58.692
    3 theta  Fixed            0.000            76.512
    4   u_x   Free            0.000             0.000
    5   u_y  Fixed            0.000           121.467
    6 theta   Free           -0.002             0.000
    7   u_x   Free            0.000             0.000
    8   u_y  Fixed          -45.000           130.555
    9 theta   Free           -0.009             0.000
   10   u_x   Free            0.000             0.000
   11   u_y  Fixed          -15.000            49.287
   12 theta   Free            0.033             0.000
```

In [293]:
```
# -------------------------
# Display Element Results
# -------------------------
# Compute and print the element-level results for each member.

print_element(1, u_global, m1, T1, k1, Qf1, disp_in_mm=True, dec=1, rad_
print_element(2, u_global, m2, T2, k2, Qf2, disp_in_mm=True, dec=1, rad_
print_element(3, u_global, m3, T3, k3, Qf3, disp_in_mm=True, dec=1, rad_
```

```
E1
u [mm,rad]: [0.0, 0.0, 0.000000, 0.0, 0.0, -0.001954]
v [mm,rad]: [0.0, 0.0, 0.000000, 0.0, 0.0, -0.001954]
q [kN,kN·m]: [0.0, 58.7, 76.5, 0.0, 61.3, -87.0]

E2
u [mm,rad]: [0.0, 0.0, -0.001954, 0.0, -45.0, -0.009059]
v [mm,rad]: [0.0, 0.0, -0.001954, 0.0, -45.0, -0.009059]
q [kN,kN·m]: [0.0, 60.2, 87.0, 0.0, 59.8, -85.7]

E3
u [mm,rad]: [0.0, -45.0, -0.009059, 0.0, -15.0, 0.032563]
v [mm,rad]: [0.0, -45.0, -0.009059, 0.0, -15.0, 0.032563]
q [kN,kN·m]: [0.0, 70.7, 85.7, 0.0, 49.3, 0.0]
```

In [296]:
```
# -------------------------
# Print Support Reactions
# -------------------------

print(np.round(f_global_complete[dof_restrained_1based - 1], 3))
```

```
[  0.     58.692  76.512 121.467 130.555  49.287]
```



Wrap-Up

Support settlement introduces **prescribed displacements at restrained DOFs**:

$$u_r \neq 0$$

The DSM equilibrium equation for the free DOFs becomes

$$K_{ff}u_f = F_f - F_f^F - K_{fr}u_r$$

The term $K_{fr}u_r$ represents the **forces induced at the free DOFs by the imposed support movements**.

Thus, support settlement enters the system as an **equivalent joint load**

$$F_f^S = K_{fr}u_r$$